

Übersetzung und Analyse von Programmen

The background is a stylized, low-poly illustration. It features a multi-story building with blue and orange architectural elements. In the foreground, there are two green trees with yellow foliage and a large, prominent yellow flower with a dark center. A bright yellow sun is visible in the upper right corner of the sky, which is composed of various shades of blue and green geometric shapes.

Vorlesung im Wintersemester 2025/26

Prof. Dr. Stephan Diehl

Informatik, Universität Trier

Willkommen in der ersten Reihe

- Heute teilweise ziemlich kleine Fonts in den Folien



Heute

- Die Java Virtual Machine
- Übersetzung von Java

Quelle: *A Formal Introduction to the Compilation of Java*, Stephan Diehl, in "Software - Practice and Experience", Vol. 28(3), 1998.
[Diehl98]

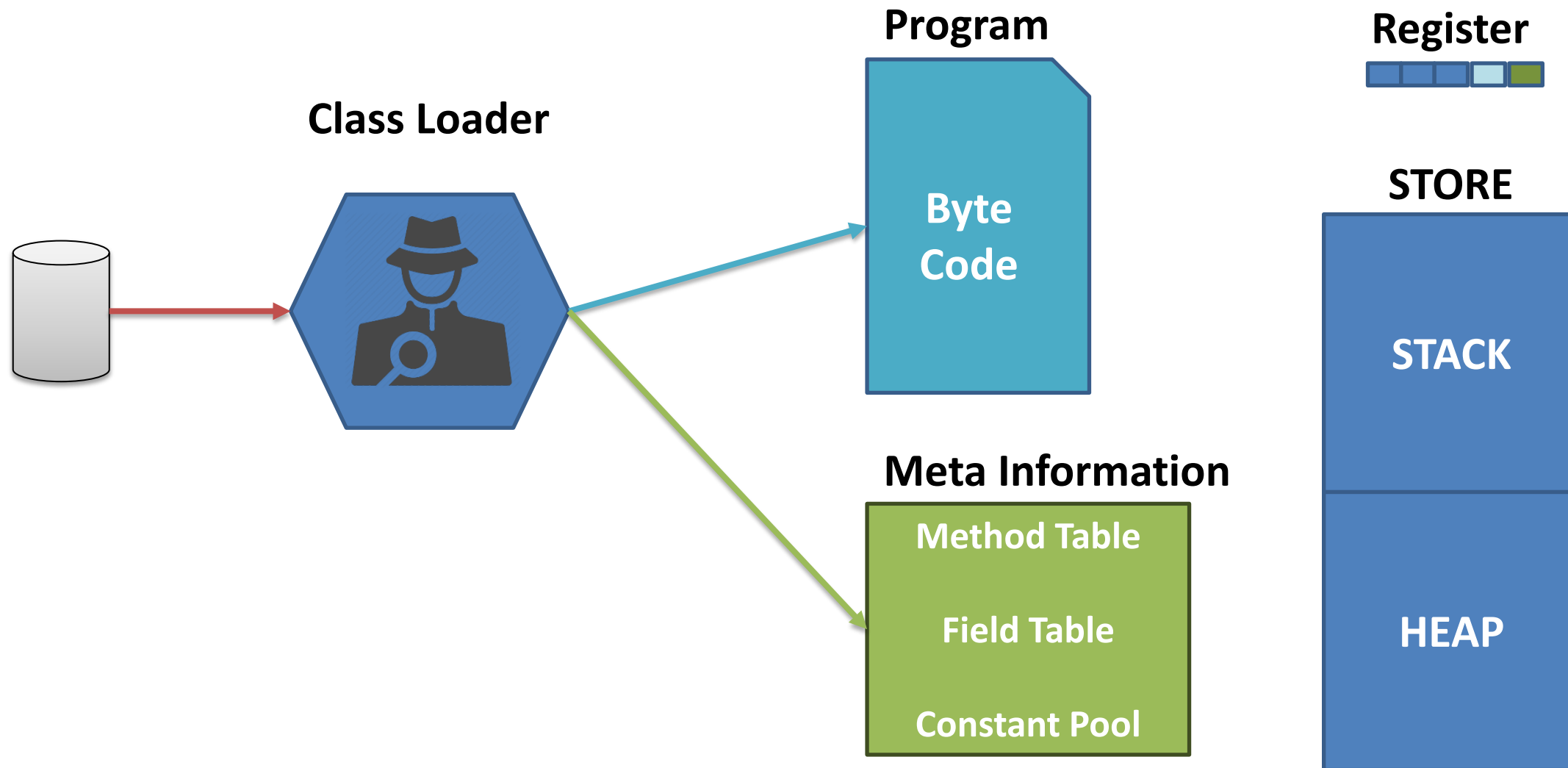


2. Teil

Übersetzung von Klassen und Methoden

Java Virtual Machine (JVM)

Komponenten

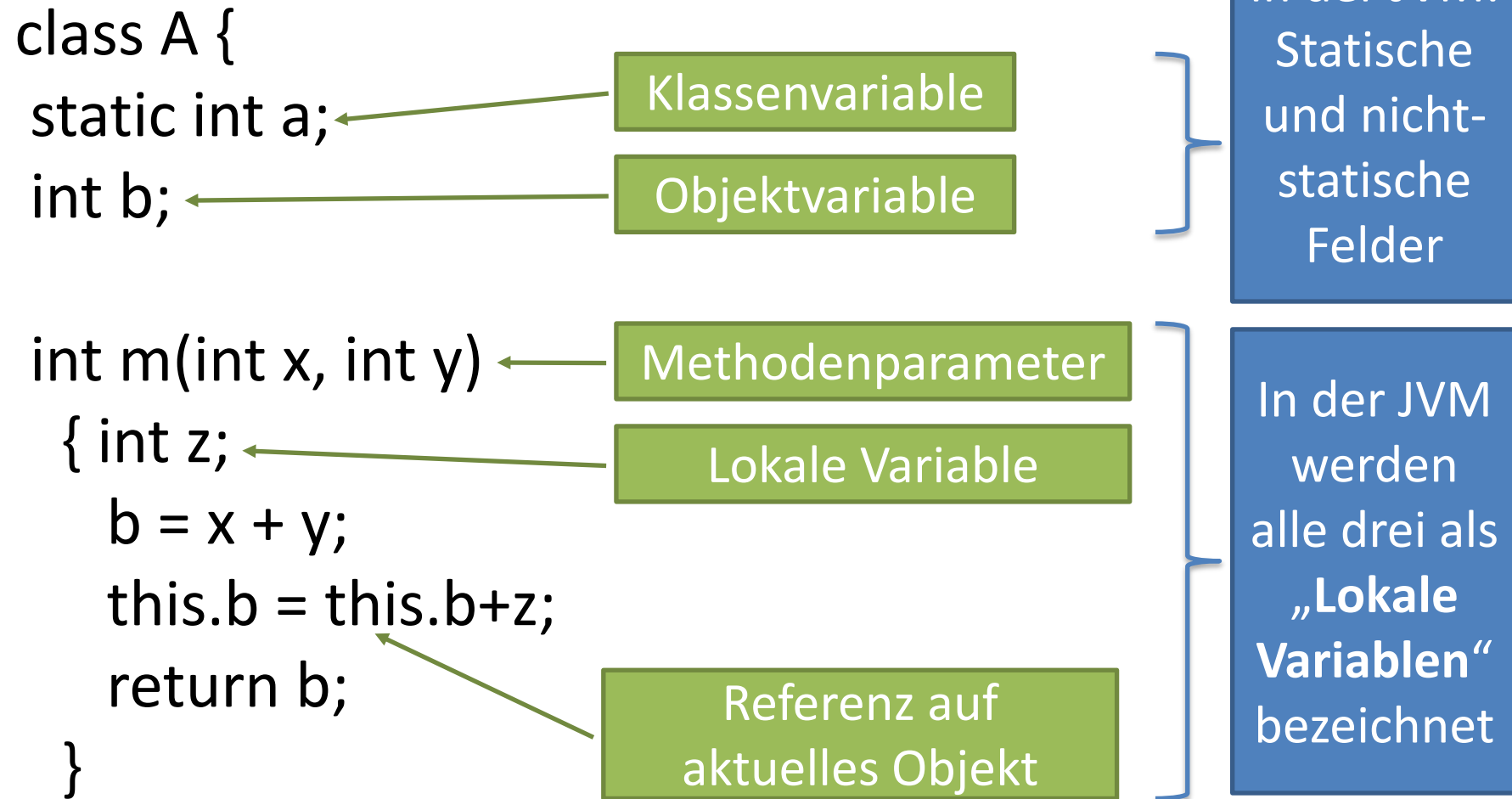


Übersetzung von Klassen

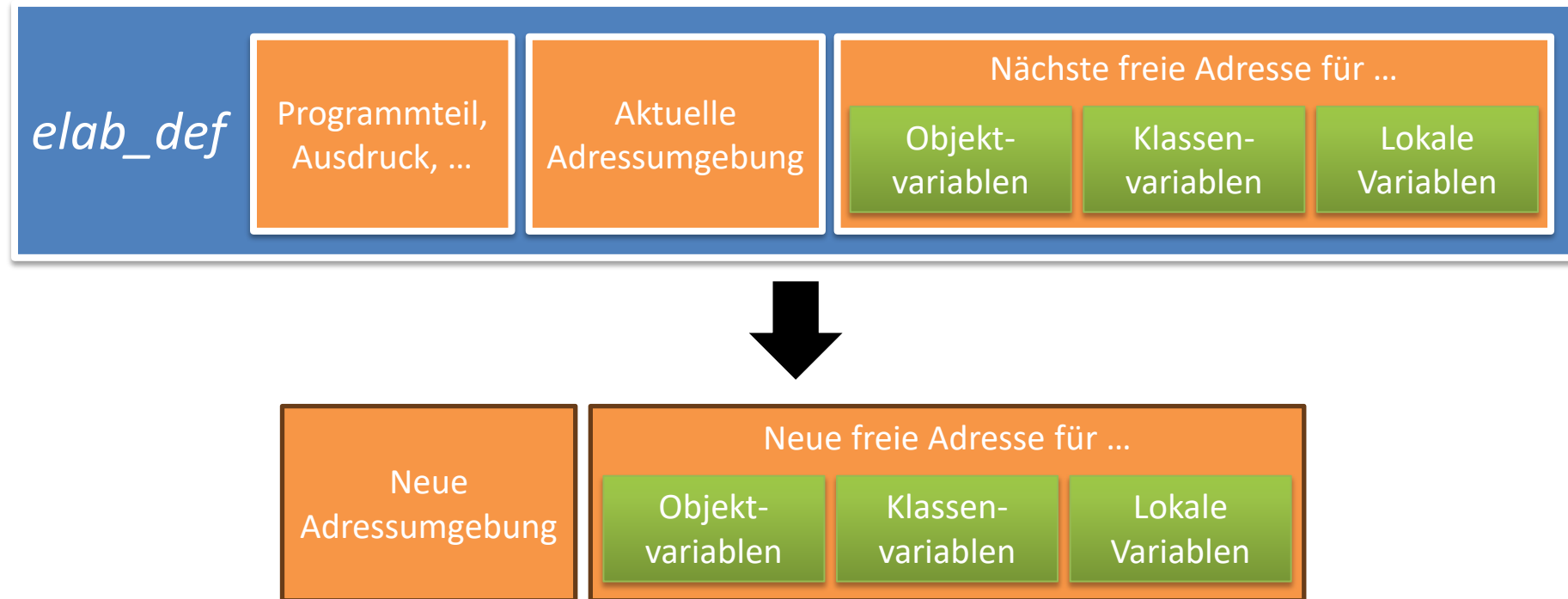
Drei Phasen

1. Berechnung der Adressumgebung:
elab_def()
2. Typinferenz für Ausdrücke und Auflösung der Überladung:
infer_type()
3. Kodeerzeugung: *code()*

Arten von Variablen in Java



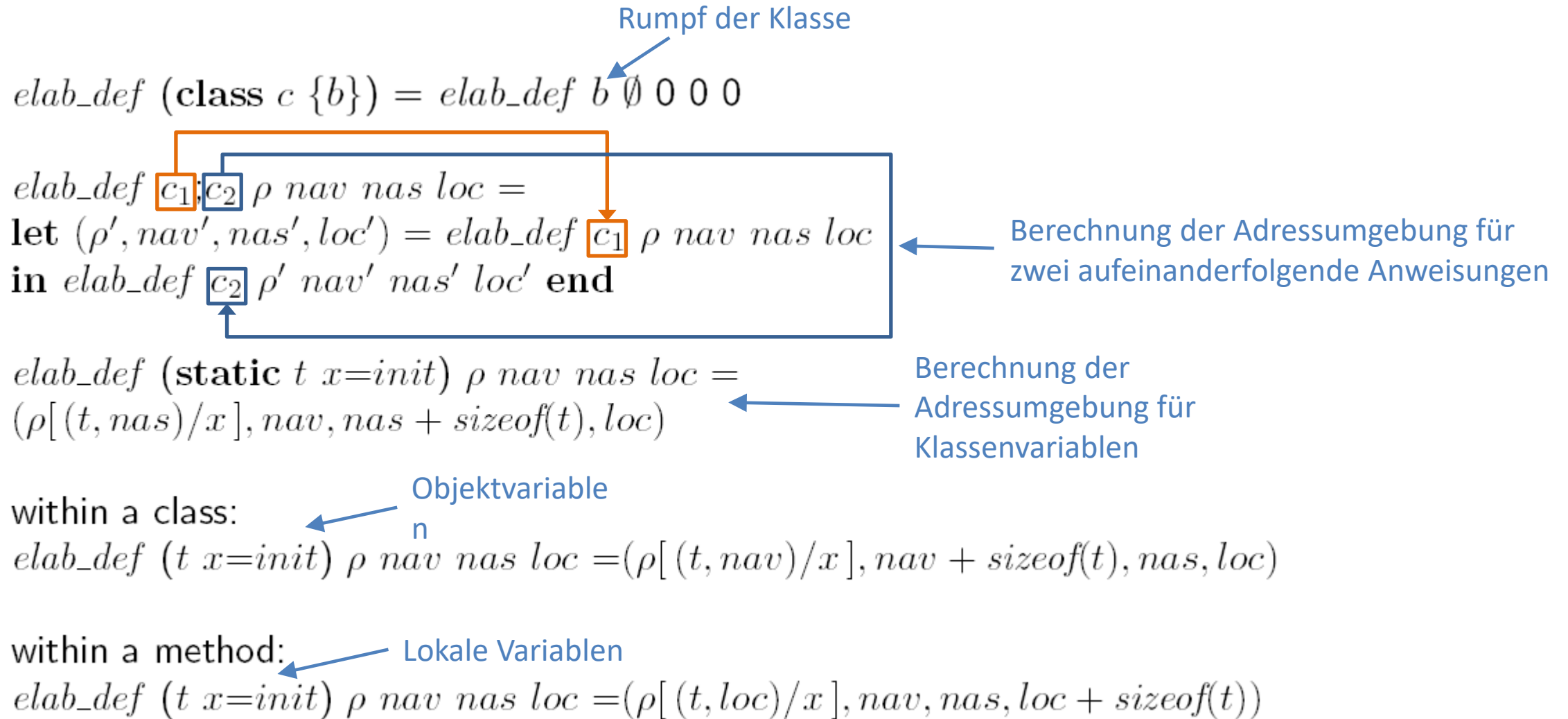
Berechnung der Adressumgebung



elab_def **b** ρ nav nas loc = (ρ' , nav', nas', loc')

Berechnung der Adressumgebung

Berechne Typen und Offsets für lokale Variablen, Klassen- und Objektfelder.



Berücksichtigung der Definitionen in der Oberklasse

Let (ρ, nav, nas, loc) be the result produced by *elab_def* for the definition of the superclass s , then the address environment for the **subclass** c is

$$elab_def(\text{class } c \text{ extends } s \{b\}) = elab_def\ b\ \rho\ nav\ nas\ loc$$

For object and class variables, we assumed on the previous slide that $\rho(x)$ is undefined, i.e. the variable extends the class, but does not overwrite a previous declaration of the variable. To take care of overwriting, we change the definition of *elab_def* as follows:

$$\begin{aligned} &elab_def(\text{static } t\ x = init)\ \rho\ nav\ nas\ loc = \\ &\quad \text{if } \rho(x) \text{ is undefined then } ((\rho[(t, nas)/x]), nav, nas + size(t), loc) \\ &\quad \text{else let } (t', nas') = \rho(x) \text{ in } ((\rho[(t, nas')/x]), nav, nas, loc) \text{ end} \end{aligned}$$

Adresse wird aus der Oberklasse
übernommen.

Berechnung der Adressumgebung

Berechne für jede Methode deren Ergebnistyp, Anzahl lokaler Variablen, und lokale Umgebung. Jede Methode wird durch ihren Namen und die Argumenttypen identifiziert.

Realistischer mit sizeof(t_i)

Berechnung der Adressumgebung
für statische Methoden

$elab_def$ (**static** t $m(t_1 p_1, \dots, t_k p_k) \{b\}$) ρ nav nas $loc =$
let $(\rho', nav', nas', loc') = elab_def$ b $\rho[(t_i, i-1)/p_i]$ 0 0 k
in $(\rho[(t, loc', \rho')/m(t_1, \dots, t_k)], nav, nas, loc)$ **end**

Berechnung der Adressumgebung
für Objektmethoden

$elab_def$ (t $m(t_1 p_1, \dots, t_k p_k) \{b\}$) ρ nav nas $loc =$
let $(\rho', nav', nas', loc') = elab_def$ b $\rho[(t_i, i)/p_i]$ 0 0 $k + 1$
in $(\rho[(t, loc', \rho')/m(t_1, \dots, t_k)], nav, nas, loc)$ **end**

Nächste freie lokale Adresse bei
 $k+1$, da eine Referenz auf das
Objekt selbst gespeichert wird

For all other cases we have: $elab_def$ $(-)$ ρ nav nas $loc = (\rho, nav, nas, loc)$

In allen anderen Fällen werden keine
Veränderungen vorgenommen

Informationen in γ_K , γ_F und γ_M

Hinzufügen von Informationen über Variablen und Methoden zum Konstantenpool γ_K , zur Feldertabelle γ_F und zur Methodentabelle γ_M .

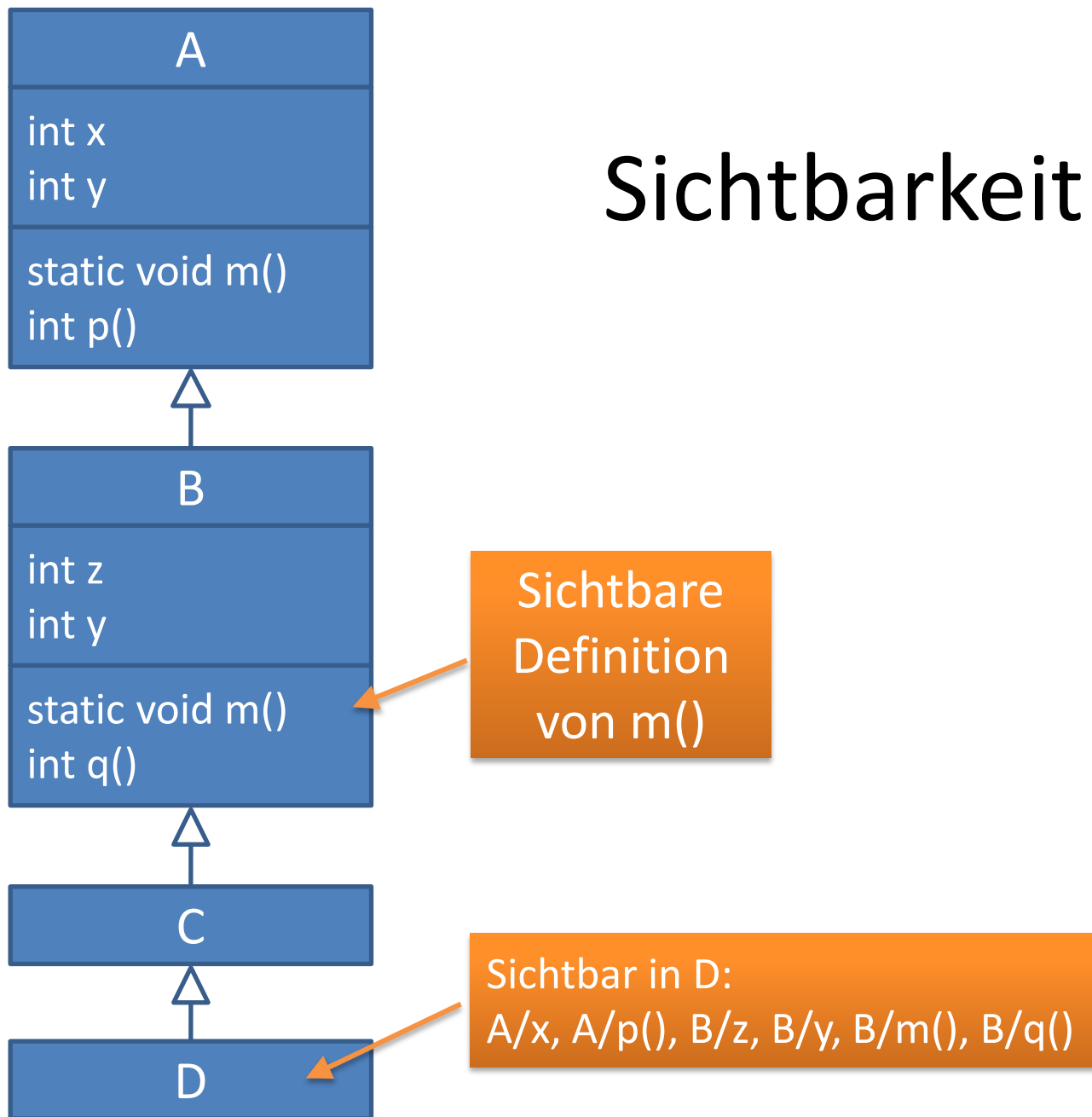
- Klassenvariablen
 - $\gamma_K(i) = "c/x"$,
 - $\gamma_F("c/x") = (t, \text{static}, a)$
- Objektvariablen
 - $\gamma_K(i) = "c/x"$,
 - $\gamma_F("c/x") = (t, \text{dynamic}, a)$
- Statische Methoden
 - $\gamma_K(i) = "c/m(t_1, \dots t_k) t"$
 - $\gamma_M("c/m(t_1, \dots t_k) t") = (c, p, n, l, \text{static}, ms)$
- Objektmethoden
 - $\gamma_K(i) = "c/m(t_1, \dots t_k) t"$
 - $\gamma_M("c/m(t_1, \dots t_k) t") = (c, p, n, l, \text{dynamic}, ms)$

Sichtbarkeit

- In einer Methode *m* einer Klasse *c* gelten die folgenden Sichtbarkeitsregeln:
 - Lokale Variablen von *m*
 - < Felder von *c*
 - < Felder der Oberklasse von *c*
 - < ...
 - Methoden von *c*
 - < Methoden der Oberklasse von *c*
 - < ...
 - Klassen im gleichen Package
 - < importierte Klassen

```
class A { int a }  
  
class B extends A {  
    static int a;  
    int b;  
  
    m(int a) {  
        return a*b;  
    }  
}
```

Sichtbarkeit und Vererbung



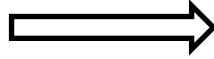
Darstellung von Typen in der JVM Konstantentabelle

LTest;	% type of an object: new Test
Test/foo(I)V	% void foo(int x)
Test/foo(I)I	% int foo(int x)
Test/foo(II)I	% int foo(int x, int y)
Test/foo(LTest;I)LTest;	% Test foo(Test o, int x)
Test/foo(LTest;LTest;)I;	% int foo(Test o1, Test o2)
[I	% one-dimensional array of integers: new int []
[[I	% two-dimensional array of integers: new int [] []
[LTest;	% one-dimensional array of objects: new Test []
Test/foo([I[[II)[LTest;	% Test[] foo(int x[], int y[][], int z)

```

public class A {
    int x=1, y=1;
    static void m() { System.out.println("m in A"); }
    int p() { System.out.println("p in A");
        return 1; }
    public static void main(String argv[]) {
        D d=new D();
        d.r(d); }
}

```



Ausgabe:

```

    m in B
    m in A
    p in A

```

```

class B extends A {
    int z=2, y=2;
    static void m() { System.out.println("m in B"); }
    int q() { System.out.println("p in B");
        return 2; }
}

```

```

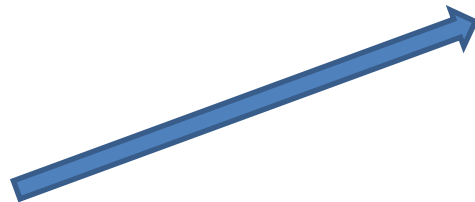
class C extends B { }

```

```

class D extends C {
    void r(A a) { C.m();
        A.m();
        int v = a.p(); }
}

```



```

void r(A);
Code:
    0: invokestatic C.m:()V
    3: invokestatic A.m:()V
    6: aload_1
    7: invokevirtual A.p:()I
   10: istore_2
   11: return

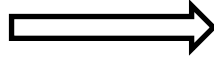
```



```

public class A {
    int x=1, y=1;
    static void m() { System.out.println("m in A"); }
    int p() { System.out.println("p in A");
        return 1; }
    public static void main(String argv[]) {
        D d=new D();
        d.r(d); }
}

```



Ausgabe:

```

m in B
m in A
p in B

```

```

class B extends A {
    int z=2, y=2;
    static void m() { System.out.println("m in B"); }
    int q() { System.out.println("p in B");
        return 2; }
    int p() { System.out.println("p in B");
        return 2; }
}

```

```

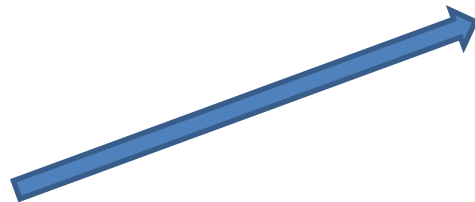
class C extends B { }

```

```

class D extends C {
    void r(A a) { C.m();
        A.m();
        int v = a.p(); }
}

```



void r(A);

Code:

0: invokestatic C.m:()V

3: invokestatic A.m:()V

6: aload_1

7: invokevirtual A.p:()I

10: istore_2

11: return

← wie zuvor !

Methodensignaturen

- Eindeutigkeit von Methoden wird über deren Signatur erreicht.
- Schreibweise: $c/m(t_1 \dots t_n)t$ mit:
 - Klassenname c
 - Methodename m
 - Typen der Parameter $t_1 \dots t_n$
 - Rückgabetyp t
- Signatur als String im Konstantenpool nachschlagbar
- Mit der Signatur können weitere Informationen aus der Methodentabelle erfragt werden

Vererbungsrelation

Definitionen

- $c \downarrow d$ iff c inherits directly from d
- $c \downarrow^0 c$
- $c \downarrow^1 d$ iff $c \downarrow d$
- $c \downarrow^n e$ iff $c \downarrow^{n-1} d$ and $d \downarrow e$
- $c \downarrow^* d$ if $c \downarrow^n d$
- $c \downarrow^+ d$ if $c \downarrow^n d$ and $n > 0$

Definierte Methoden und Felder

- $methods(c) = \{m(t_1 \dots t_k)t \mid \text{class } c \text{ contains a definition of a method with signature } m(t_1 \dots t_k)t\}$
- $fields(c) = \{(v, t) \mid \text{class } c \text{ contains a definition of a field } v \text{ of type } t\}$

Einschränkung für Methodenüberschreibungen:

- $\forall m(t_1 \dots t_k)t \in methods(c) : \forall d : c \downarrow^+ d :$
 $\forall m(t_1 \dots t_k)t' \in methods(d) : t \sqsubseteq t'$
 relation $\sqsubseteq$ (called “more specific”)

```
class A {
    m() { return new B(); }
}
```

```
class B extends A {
    m() { return new A(); }
    int b;
}
```

```
B b = new B();
b.b
```

Sichtbare Definitionen

$$\begin{aligned}
 vis_meths(c) = \{ & d/m(t_1 \dots t_k)t \mid \\
 & \exists n : m(t_1 \dots t_k)t \in methods(d) \text{ and } c \downarrow^n d \\
 & \Rightarrow \underbrace{\nexists n', d' : n' < n, m(t_1 \dots t_k)t' \in methods(d') \text{ and } c \downarrow^{n'} d'}_{n \text{ ist minimal}} \}
 \end{aligned}$$

vis_meths(c) enthält alle Methoden von c, da n = 0 sein kann

$$\begin{aligned}
 vis_fields(c) = \{ & (d/v, t) \mid \\
 & \exists n : (v, t) \in fields(d) \text{ and } c \downarrow^n d \\
 & \Rightarrow \underbrace{\nexists n', d' : n' < n, (d'/v, t') \in fields(d') \text{ and } c \downarrow^{n'} d'}_{n \text{ ist minimal}} \}
 \end{aligned}$$

vis_fields(c) enthält alle Felder von c, da n = 0 sein kann

Typinferenz

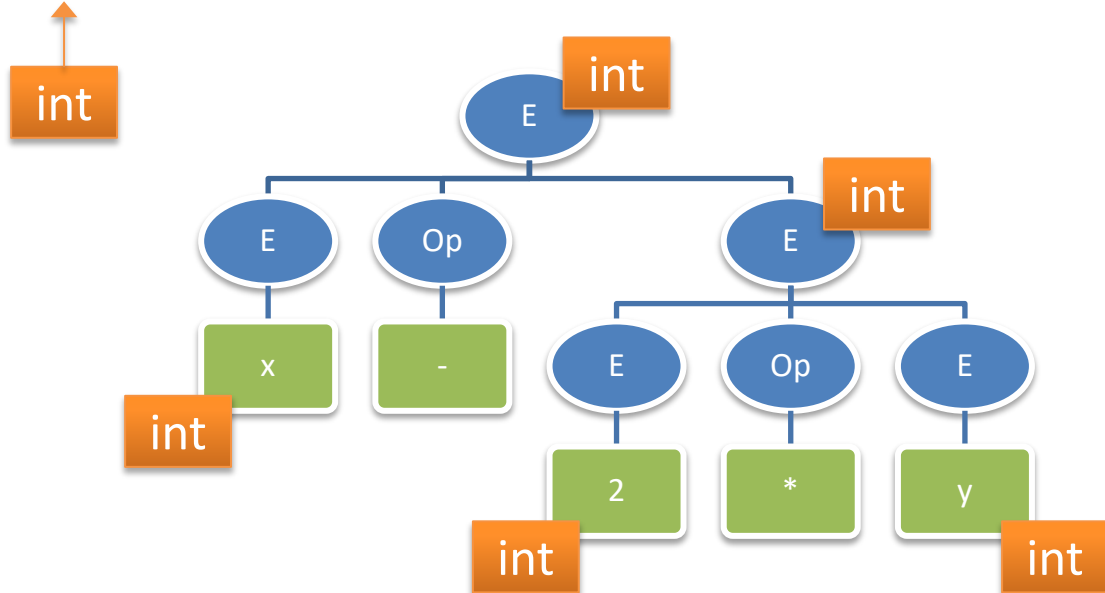
Definition

- Typinferenz berechnet den Typ eines Ausdrucks bottom-up, d.h. ausgehend von den Typen der Blätter (Variablen und Konstanten) des Ausdrucksbaums.

Beispiel:

int x, y;

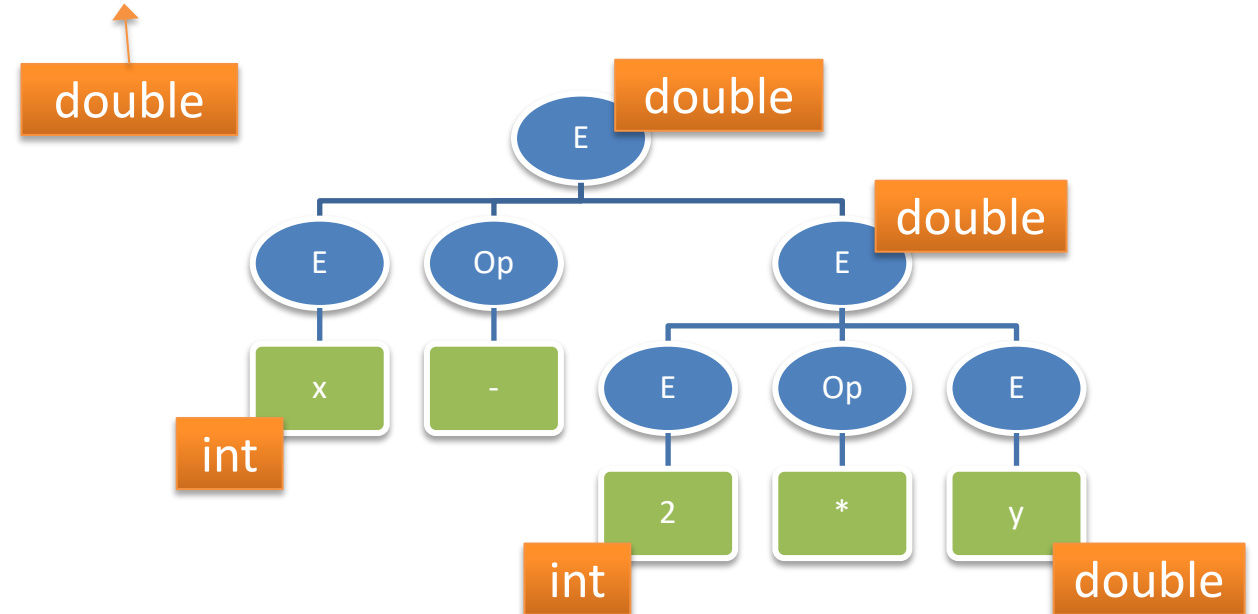
z = x-2*y;



Beispiel:

int x; **double** y;

z = x-2*y;



Typinferenz

Anwendung

- Codeerzeugung eines Methodenaufrufs
 - Welche Methode soll aufgerufen werden?
 - **Aufruf:** `test(x+y)`
 - **2 Deklarationen:** `void test(int i)` vs. `void test(String s)`
- Ableiten der Typen aller Argumente (bottom-up)
 - Verwenden der Funktion *infer_type()*



Typinferenz

Konstanten und
lokale Variablen

$\text{infer_type } c \ \rho = t$ where c is a constant of type t
 $\text{infer_type } x \ \rho = t$ where x is a local variable and $(t, a) = \rho(x)$

Binäre Ausdrücke

$\text{infer_type } (e_1 \text{ op } e_2) \ \rho = t_1 \tilde{\text{op}} t_2$

where $t_1 = \text{infer_type } e_1 \ \rho$, $t_2 = \text{infer_type } e_2 \ \rho$ and $\tilde{\text{op}}$ is the abstract operation on types induced by op .

(mehrdimensionale) Arrays

$\text{infer_type } (e[e_1] \dots [e_k]) \ \rho = t'$

where e yields an array and $(m, t) = \text{infer_type } e \ \rho$,

and $t' = \begin{cases} t & \text{if } m = k \\ (m - k, t) & \text{if } m > k \end{cases}$

Liefert Anzahl der
Dimensionen m
und den Typ t

$\vdash : \text{int} \times \text{int} \rightarrow \text{int}$
 $\tilde{\vdash} : \text{Typ} \times \text{Typ} \rightarrow \text{Typ}$
 $\tilde{\vdash}(\text{int}, \text{int}) = \text{int}$
 $\tilde{\vdash}(\text{int}, \text{double}) = \text{double}$

Typinferenz

$\text{infer_type } o.x \ \rho = t$

where x is field, $\text{infer_type } o \ \rho = t_0$

and $(d/x, t) \in \text{vis_fields}(t_0)$

$\text{infer_type } x \ \rho = t$

as above, but $t_0 = c$ where c is the current class

$\text{infer_type } c.x \ \rho = t$

as above, but $t_0 = c$

$\text{infer_type } o.m(e_1, \dots, e_k) \ \rho = t$

where $t_0 = \text{infer_type } o \ \rho$, $t_1 = \text{infer_type } e_1 \ \rho$, $\dots$, $t_k = \text{infer_type } e_k \ \rho$ and $d/m(t_1 \dots t_k)t \in \text{vis_meths}(t_0)$

$\text{infer_type } m(e_1, \dots, e_k) \ \rho = t$

as above, but $t_0 = c$ where c is the current class

$\text{infer_type } c.m(e_1, \dots, e_k) \ \rho = t$

as above, but $t_0 = c$

$\text{infer_type } o.m(e_1, \dots, e_k) \ \rho = \boxed{t}$

where $t_0 = \text{infer_type } o \ \rho$, $t_1 = \text{infer_type } e_1 \ \rho$, $\dots$, $t_k = \text{infer_type } e_k \ \rho$ and $d/m(t_1 \dots t_k)\boxed{t} \in \text{vis_meths}(t_0)$

Für Methodenaufrufe benötigen wir die Typen der Argumentausdrücke, dann können wir die Signatur in den sichtbaren Definitionen nachschlagen und den Ergebnistyp bestimmen.

Übersetzung mehrdimensionaler Arrays

$\text{code } \mathbf{e}[i_1] \dots [i_k] \rho =$

```

code e ρ;
code i1 ρ;
aaload;
:
code ik-1 ρ;
aaload;
code ik ρ;
iaload;

```

Where $\text{infer_type}(\mathbf{e}) = (m, t)$, $t = \text{integer}$ and $m = k$.

If $m > k$ we use **aaload** instead of **iaload**.

We also use **aaload** if t is a classname and $m = k$

Überladen von Methoden

- Überladen von Methoden
 - Verwenden von gleichen Methodennamen für zwei unterschiedliche Methoden
 - Unterschiedliche Parametertypen
 - Methoden dürfen sich nicht ausschließlich im Rückgabewert unterscheiden
- Beispiel:

```
class Transform {  
    void scale(Polygon p, double d) {...}  
    void scale(Polygon p, double d, Vect2D v) {...}  
    void scale(Circle p, double d) {...}  
    boolean scale(Polygon p, double d) {...}  
}
```



Auflösen von Überladung

```
class Transform {
    void scale(Polygon p, double d) {...}
    void scale(Polygon p, double d, Vect2D v) {...}
    void scale(Circle p, double d) {...}
}
```

- Einfacher Fall:
 - Nach Ableiten der Argumenttypen des Methodenaufrufs mittels *infer_type()*, kann die entsprechende Methode in γ_M gefunden werden.

- **Beispiel:**

`scale(p, d)`

→ *infer_type()* liefert für `p` den Typ `Polygon` und für `d` den Typ **double**

- Komplexer Fall:

→ `scale(Polygon, double) ?`

- Es gibt keinen exakten Fund.

- **Beispiel:**

`scale(t, 2)`

→ *infer_type()* liefert für `t` den Typ `Triangle` und für `2` den Typ **int**

Triangle ↓⁺ Polygon

→ `scale(Triangle, int) ?`

int kann ohne Verlust nach
double konvertiert werden

Auflösung von Überladung

relation $\sqsubseteq$ (called “more specific”)

- $t \sqsubseteq t'$ iff t, t' are classes and $t \downarrow^* t'$
- $t \sqsubseteq t'$ iff t, t' are primitive types and entities of type t can be converted into entities of type t' without data loss.
- $m(t_1 \dots t_k) \sqsubseteq m(t'_1 \dots t'_k)$ iff $t_i \sqsubseteq t'_i$ ($1 \leq i \leq k$)

Anwendbare Methodendefinitionen:

- $applicable(c/m(t_1 \dots t_k)) = \{c'/m(t'_1 \dots t'_k)t' \in vis_meths(c) \mid m(t_1 \dots t_k) \sqsubseteq m(t'_1 \dots t'_k)\}$

Methode muss in dieser Form
nicht unbedingt existieren

Menge von anwendbaren Methodendefinitionen

- $most_specific(A) = c/m(t_1 \dots t_k)t \in A$ such that
 $\forall c'/m(t'_1 \dots t'_k)t' \in A : m(t_1 \dots t_k) \sqsubseteq m(t'_1 \dots t'_k)$

Typinferenz für überladene Methodenaufrufe

$$\text{infer_type } o.m(e_1, \dots, e_k) \ \rho = t'$$

$$\text{where } t_0 = \text{infer_type } o \ \rho,$$

$$t_1 = \text{infer_type } e_1 \ \rho, \dots, t_k = \text{infer_type } e_k \ \rho \text{ and}$$

$$d/m(t'_1 \dots t'_k)t' =$$

$$\text{most_specific}(\text{applicable}(t_0/m(t_1 \dots t_k)))$$

Beispiel

```
class A {}
```

```
class B extends A {}
```

```
class C extends B
```

```
① { void m(A x, C y) { }
```

```
②   void m(B x, A y)  
      { m(new B(), new C());  
      }  
}
```

applicable(C/m(B,C) = ?

most_specific(applicable(C/m(B,C)) = ?

Beispiel

```
class A {}
class B extends A {}
class C extends B
{ void m(A x, C y) { }
  void m(B x, A y)
    { m(new B(), new C());
    }
}
```

For the arguments of the method call we infer types B and C and thus get $\text{applicable}(C/m(B, C)) = \{m(A, C), m(B, A)\}$ because $m(B, C) \sqsubseteq m(A, C)$ and $m(B, C) \sqsubseteq m(B, A)$. Unfortunately neither is $m(A, C) \sqsubseteq m(B, A)$ nor $m(B, A) \sqsubseteq m(A, C)$ and thus the call is ambiguous .

Hinweis: Die Definitionen von $m()$ sind zulässig, der Aufruf ist das Problem.

Zum Beispiel ist $m(\text{new } B(), \text{new } B())$ möglich: $\text{invokevirtual } C.m: (LB; LA;) V$